CE/CS/SE 3354 · Software Engineering · Fall 2026

Project Milestone P2

Analysis and High-Level Design

Cooket

Team	Team 01, section .005
Document version	1.0 (baseline for P2; grows from your P1 version 0.x)
Date	11/02/2026
Repository URL	https://github.com/zxahamza20/CS3354-Project-Cooket
Git tag for this milestone	p2 (points at the commit that contains this exact PDF)
Submitted on Canvas by	--

Team member	NetID	GitHub username
Hamza Munis	Hxm240022	zxahamza20
Gabriel Aguilar-Vega	gja230002	gabev178
David Vu	dmv160030	DavidWho0708
Siddharth De	SXD230138	Siddking2
Raagavika Govindarajan	Rxg240015	raagavikagovindarajan

HOW TO USE THIS TEMPLATE

This is not a new document. P2 continues the document you baselined at P1. Copy your P1 sections 1 to 3 into this file (updated under change control), then fill in the new sections 4 to 9. Your P1 project plan, open questions and baseline move to the end and are renumbered:

P1 section 4 (project plan) becomes section 10. P1 section 5 (open questions) becomes 11. P1 section 6 (baseline) becomes 12.

Delete every guidance, example and warning box (blue, amber and red) and the two marking pages at the end before you submit. They are instructions, not content. Boxes or placeholders such as "[TBD]", "PR #[n]" or "[Member 3]" left in the submitted document lose document-quality points.

Length: sections 4 to 9 should add roughly 12 to 18 pages, diagrams included. Every section below carries a budget. A longer document is not a better one.

Diagrams: draw them in any tool (draw.io, Lucid, PlantUML, or a tidy hand drawing that is scanned). Export each as an image for this document and commit the source file to /docs/p2/diagrams. A diagram that cannot be read cannot be credited for its content.

Required structure: using this file is optional, but its section numbering is required, because the rubric refers to it.

Rubric: the marking pages at the end show where the 60 points sit. Read them before you divide the work.

WHAT P2 DOES NOT ASK FOR (DELETE THIS BOX BEFORE SUBMITTING)

P2 is analysis and high-level design. At this milestone we are NOT asking for:

design class diagrams with operations, parameter types or visibility, and design-level sequence diagrams with operation signatures (Chapter 9 detailed design; they enter the document during P3 work)

production code (the only code in P2 is the small architecture spike in section 8)

a t12est plan (P3, after Chapter 11)

screen mock-ups or a visual design (not graded at this milestone; add them only if they settle an open question)

design patterns from the Gang of Four catalog, ATAM or utility trees (not required; do not spend your time there)

Work placed in the wrong milestone is not credited. Spend the time on sections 4 to 9.

0 Revision History, Contribution Record and Changes Since P1

All three tables below are graded. The contribution record carries the individual 20% of this milestone and must agree with your repository history.

Version	Date	Author	Summary of change
0.x			P1 baseline (copy your last P1 row)
1.0			P2 baseline: analysis and high-level design added

0.1 Individual contribution record (P2)

Team member	Sections and artifacts this person produced for P2	Evidence in the repository (commit hashes, PR numbers)	Reviews given (PR numbers)
David Vu	10.4 Repository; Repository Folder Structure (/docs/p1, /docs/p2, /docs/p2/diagrams, /spikes/S-01); 10.3 Risks; 11 Open Questions; 0.2 Changes Since P1 (CH-01 and CH-02); Ingredient Plural Fix in score.js; 6.1 Architectural Drivers	Commits: 739ed38 (docs/p1), b1656b3 (docs/p2), 809b901 (spikes/S-01), 05da72b (docs/p2/diagrams) PR: #1 (10.4 Repository); #8 (10.3 Risks); #14 (score.js Ingredient Plural Fix)	
Hamza Munis	4.1 repository, 4.2 & 4.3 repository, frontend, including ingredient input, strict/flexible search modes, optional preferences, backend scraper API integration, and recipe results with loading and error handling.	Commits: 619c54f (frontend/all files), 3992897 (docs/p2), 9b93639 (docs/p2), 83abd32 (docs.p2), d38b3cc (doc/p2/diagrams, doc/p2), PR: #12 (4.2 section repo), #13 (4.3 section repo)	8, 9, 14
Gabriel Aguilar-Vega	Sections 5 & 8, Section 0.2 CHO-3 changes		

Team member	Sections and artifacts this person produced for P2	Evidence in the repository (commit hashes, PR numbers)	Reviews given (PR numbers)
Siddharth De	8 Prototype Evidence: Architecture Spike S-01 (/spikes/S-01: measure.js and README with results); data storage (database integration): Supabase recipes table (backend/db/schema.sql), database client and recipe store (backend/db/client.js, backend/db/recipes.js), checking stored recipes before scraping in the search route (backend/routes/webscrape.js), database documentation (docs/Database.md); 6.4 deployment diagram source and image (/docs/p2/diagrams)	Commits: cc65f67 (schema, client, recipe store), 78145be (stored recipes checked before scraping), 744fb1b (docs/Database.md, spike S-01), ec44490 (6.4 deployment diagram) PR: #16 (database branch)	10

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Evidence means something a reader can open and that existed before the deadline: a commit hash, a pull request, a file in the repository. Issues you own count as indirect evidence; link the commits that close them for full credit.

Commit your own work from your own GitHub account. If one person uploads everyone's text from a shared document, the history shows one author and the other members have no verifiable evidence. A shared drafting document is fine; the final text of each section is committed by its owner.

Cite only what exists. Every commit, pull request, branch and file named in this table is checked. Citing evidence that is not in the repository is an academic-honesty matter, not a formatting slip.

Every member appears with a distinguishable contribution. Two rows with identical wording are marked as vague claims. If a member did not contribute, write that plainly in their row rather than leaving it blank or covering for them.

0.2 Changes since the P1 baseline

Change ID	What changed (IDs affected)	Why (P1 feedback, new finding, spike result)	Knock-on effect	Agreed on
CH-01	Scope 1.2: "Automatically buy new ingredients" is now moved to out of scope	It had contradicted the payments exclusion in the constraints	Scope now matches the payment constraint in 2.4	Yes

Change ID	What changed (IDs affected)	Why (P1 feedback, new finding, spike result)	Knock-on effect	Agreed on
CH-02	Repository: P1 PDF, diagram sources and each member's sections committed from their own accounts	P1 Feedback: C4, the repository only had the README for P1	Contribution records can now be linked to commits	Yes
CH-03	Withdrawn CON-02	Caused conflicting ideas of whether or not the user should have an account or not based on the feedback. Agreed to having an account as a requirement	Everything on the doc aligns with the requirement of having an account to search.	Yes

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Start with your P1 feedback. Every point raised in your P1 comment should appear here as a change, or with one sentence explaining why you kept the original. This table is where the baseline-and-change-control idea from Chapter 5 becomes real.

Requirement IDs are never reused. A removed requirement stays in section 3 marked "Withdrawn (CH-nn)" so the traceability matrix does not silently lose it.

Budget: half a page.

EXAMPLE (COMETPARK): DELETE BEFORE SUBMITTING

CH-02 · QR-02 rewritten as quality-attribute scenario QAS-01 · Why: P1 feedback said "fast enough" had no response measure · Knock-on: ADR-02 now cites QAS-01 · Agreed 30 Oct.

1 to 3 Introduction, Overall Description and Requirements (carried forward)

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Paste your P1 sections 1, 2 and 3 here and bring them up to date. Keep their numbering and IDs.

Fix before you build on it: any requirement that is still untestable, any source that still says "pending", any open question your P1 feedback flagged. Analysis and design inherit every ambiguity you leave in section 3.

Use cases: section 3.4 keeps the use-case diagram and the list at actor-goal level. The detailed expansion now lives in section 4; do not copy it twice.

Traceability: replace the P1 matrix in section 3.5 with a one-line pointer to section 9.1, where it is extended.

If nothing in a subsection changed, keep it and say so in one line under its heading ("Unchanged since v0.x").

1 Introduction

1.1 Purpose of this document and of the system

The purpose of this document is to define the baseline with functional and non-functional requirements for the Cooket recipe recommendation system. It lays out the technical boundaries, user roles, and operational constraints and serves as the formal design agreement between the project development team and the course instructors before the architectural and implementation decisions.

The system solves the problem of finding the right recipes, based on the available ingredients. Manual searching for recipes is very inefficient and can cause decision fatigue because it often displays a lot of non-instructional data and half the time the recipe includes ingredients which are not even available at hand, because it assumes the pantry is stocked. Currently, home cooks like students with isolated or leftover ingredients must browse multiple recipe sources, manually cross-reference ingredient lists, and guess how well a dish matches their immediate cravings. Cooket automates web recipe scraping and provides a structured recommendation system where users can query recipes by available ingredients, having a flexible or limited pantry and rank matching options across different culinary criteria such as flavor profiles, textures, and target food categories.

1.2 Scope

In scope — we will build this	Out of scope — we will not, and why
Taking inputs from users in text form	Automatically get ingredients from image
Get and filter results of web-scraping recipes	Generate new recipes
Suggest recipes based on inputs	Handle payments
Prompt the user to login or create an account if not done so.	Automatically be able to buy new ingredients for recipe

Why the exclusions were excluded:

Automatically getting ingredients from images is out of scope because it is unneeded for the project goal. We will not generate new recipes, since AI generated recipes cannot be fully trusted. We're not going to handle payments due to security risks and liability.

1.3 Definitions, acronyms and abbreviations

Term or abbreviation	Meaning as used in this project
Web Scraping	The automated process of collecting recipe information, including ingredients, instructions, and other relevant details, from websites.
Strict Ingredient Mode	A search option that only recommends recipes that can be made using the ingredients the user has specified.
Flexible Ingredient Mode	A search option that allows recipes requiring additional ingredients beyond those specified by the user.
Match Score	A value representing how closely a recipe matches the user's ingredients and preferences, used to rank recommendations.
Recipe Database	A collection of recipes and associated information retrieved from online sources for searching and recommendations.

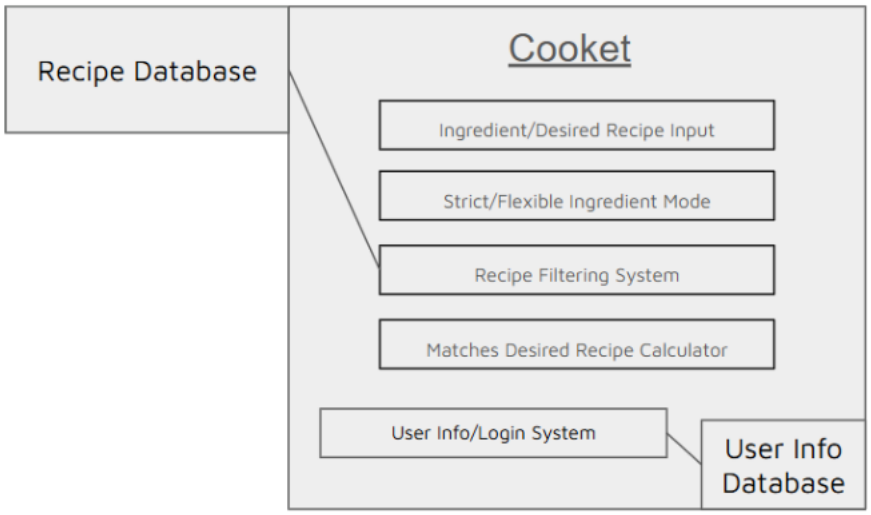
1.4 References

[R1]

No References Used here

2 Overall Description

2.1 Product perspective and context



The recipe database sits outside our system as we will have a web scraper search for recipes throughout the internet to put into the system. It's the only system we will not create with every other system/database being made/maintained by us while also getting recipes from the database if the system requires it.

2.2 Stakeholders and how you reached them

Stakeholder	Role in relation to the system	What they need from it	How you learned this
User	They use the system, and want the system to be the best, as in report any bugs or glitches and are the reason advertisers sponsor us	A working system that can get them the recipes they want based on their tastes.	Interviewed Baking club members and students interesting in cooking
Maintainer	Operates, updates, and monitors the scraper, parser, and recipe database	A system can easily be adjusted, has consistent nomenclature, easy to iterate on and deploy.	We assumed based experience as maintainer
Advertisers	The sponsors who pay for the system	The ability to promote their own product and get users to engage with it or get their information	We assumed that's what advertisers would want

2.3 User characteristics

The users are people who want to look up (and usually cook) recipes tailored to themselves. They can be assumed to know what ingredients they have. If they're wanting a specific texture or flavor, they would know that as well. They also can be assumed to know what type of food they want, if any. The only constraint on the users is that the system requires internet and access to the sites we will be web-scraping, so some locations will have worse recipe results.

2.4 Constraints

"We will not handle payments", this is a constraint due to risk of data breaches and overall estimated workload would push the project to take longer than available time.

2.5 Assumptions and dependencies

#	Assumption or dependency	What breaks if it does not hold
A1	Standard ingredient naming conventions so that ingredients input by users can be matched against the scraped recipe text	Searches will yield false negative zero match results for recipes that list synonyms or modified names
A2	Target recipe websites maintain a reasonable and consistent markup and standard schema structures during our scraping cycles	The extraction pipeline will fail to parse ingredients and instructions, producing stale, corrupt, or incomplete records which would require manual parsing for the affected sites
A3	Target culinary websites remain online, accessible over standard HTTPS, and do not enforce strict IP anti-bot/CAPTCHA firewalls against scraper requests.	Automated recipe ingestion will halt or return HTTP 403/429 status codes, preventing database refreshes and new recipe discovery.
A4	Supabase backend services and database APIs stay online and accessible over our network connection.	The app won’t be able to save or pull scraped recipes, breaking our search and recommendation tools.
A5	Our recipe database storage and query volume stay within Supabase’s free-tier limit.	Database requests might get throttled or blocked, causing recipe searches to time out or fail for users.

2.6 Feasibility summary

Feasibility gate	Verdict	Evidence behind the verdict
Technical	Feasible	3 team members have experience in ML, and one has done work with web scraping see https://github.com/Krish54491

Operational	Feasible	After doing some interviews with cooking and baking inclined students, most students said they would use a product like this, especially since it's free
Schedule	Feasible	Most of our team has experience with the technologies used and the bulk of the work will be done in the backend which is easier to test
Economic / resource	Feasible	There is no Economic / resource being used other than our laptops and time

3 Specific Requirements

3.1 Functional requirements

ID	Requirement — one "shall" sentence	Priority	Source	Acceptance criterion
FR-01	The system shall allow users to input their ingredients into the search tab	MUST	MAINTAINER	A tester can confirm this by inputting a basic list of 5 ingredients into the search tab that is understood by the system
FR-02	The system shall be able to returns multiple meal recipes made with the user's ingredients	MUST	MAINTAINER	A tester can confirm this by inputting the base ingredients for simple recipe (like a PB&J sandwich), and check if the system outputs that as a result
FR-03	The system can be able to sort by popular recipes	COULD	USER	A tester can confirm this by inputting their ingredients and getting an organized output of popular recipes
FR-04	The system includes accurate measurements for recipes in any measurement quantities	MUST	MAINTAINER	This can be tested by inputting ingredients and quantities via the users preferred measurement quantities and seeing if the output follows the same measurements
FR-05	The system should allow the user to save their favorite recipes	SHOULD	USER	A tester can confirm this by selecting the

				favorite button for a certain recipe and accessing that on the favorites tab
FR-06	The system can provide recommend recipes based on the types of foods/ingredients the user frequently has or uses	WON'T	ADVERTISER	A tester can confirm this by favoriting a few recipes, and then checking to see if the system offers any recommended recipes based on their favorites
FR-07	The system can provide a video of the recipe being made (if the recipe is from a video platform like YouTube)	COULD	USER	A tester can confirm this by selecting a recipe and checking to see if a video of it being made is linked in its description
FR-08	The system can allow you to post your own recipes that others can see/get recommended	WON'T	ADVERTISER	A tester can test this by posting their own recipe and check if they receive other user made recipes in the recommendations
FR-09	The system shall users to login or register before searching for recipes	MUST	USER	A test can check this by opening the site and seeing if it lets them search without logging in or registering

3.2 Quality requirements

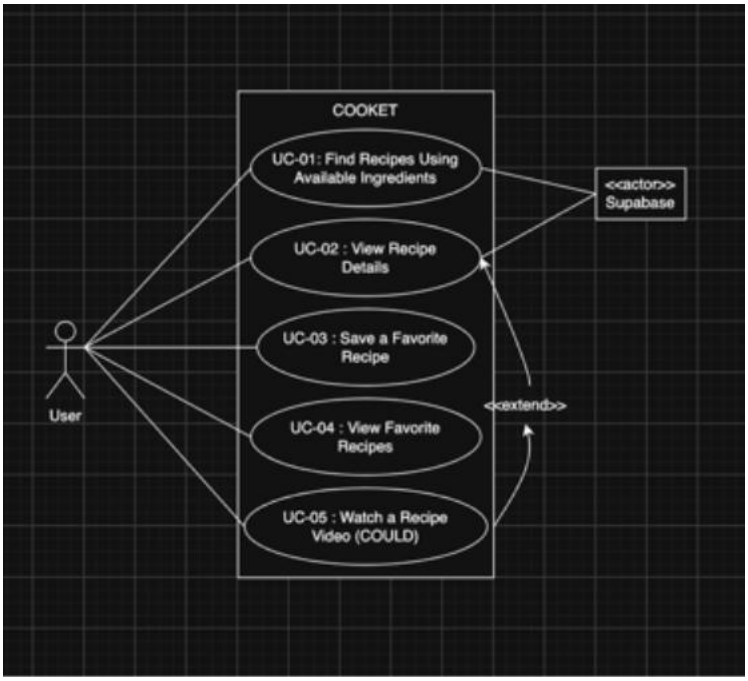
ID	Requirement — with threshold, condition and method	Priority	Source	How it will be verified
QR-01	The system shall return recipe search results within 3 seconds for at least 95% of searches containing 1–10 ingredients	MUST	USER	This can be verified by performing 100 searches using different ingredient combinations and verifying that at least 95 finishes within 3 seconds

QR-02	The system shall allow at least 90% of first-time users to complete an ingredient search without assistance within 60 seconds	SHOULD	USER	This can be verified by conducting a usability test with 10 first-time users and verify that at least 9 complete the task within 60 seconds
QR-03	The system shall correctly identify at least 90% of ingredient matches between user inputs and recipe ingredient lists in a test set of 100 ingredient comparisons	MUST	MAINTAINER	This can be verified by compare the system's ingredient matching results against 100 manually verified ingredient comparisons and calculate accuracy
QR-04	The system shall use HTTPS for 100% of communication between the user's browser and the application server	MUST	USER	This can be verified by inspecting network requests using browser developer tools and verify that all browser-to-server requests use HTTPS

3.3 Constraint requirements

ID	Constraint — one sentence	Source	Why it cannot be negotiated
CON-01	The system shall retrieve existing recipes rather than generate new recipes using AI	PROJECT SCOPE	AI-generated recipes are excluded because we want to provide recipes from existing sources rather than risk generating unreliable cooking instructions
Withdrawn (CH-03)	The system shall allow users to search for recipes without requiring account registration	PROJECT SCOPE	users should be able to access the application's primary functions without creating an account for convenience and keeping users on the app

3.4 Initial use cases



Expanded use case — copy this block for each of your two

Field	Content
Identifier and name	UC-01 - Find Recipes Using Available Ingredients
Primary actor	User
Supporting actors	Supabase database service
Preconditions — what must be true before it starts	The user can access Cooket. The recipe database is available and contains recipe records. No account is required.
Trigger — what starts it	The user submits their available ingredients to search for recipes.
Main success scenario — numbered steps	1. The user enters their available ingredients. 2. The user submits the search. 3. Cooket processes the ingredient input. 4. Cooket retrieves matching recipes from the database. 5. Cooket displays the matching recipes. 6. The user reviews the results and identifiers recipes of interest.
Alternative and exception flows	2a. Empty input: Cooket asks the user to enter ingredients before searching. 4a. No matching recipes: Cooket displays a no-results message and allows the user to revise the search. 4b. Database unavailable: Cooket reports the search could no be completed and allows the user to retry. 5a. Popularity sorting: If this COULD feature is implemented, the user can request that results be sorted by popularity.
Postconditions — what is true when it succeeds	Matching recipes have been displayed for the ingredients submitted by the user.

Requirements realized	FR-01, FR-02; FR-03 for optional popularity sorting.

Field	Content
Identifier and name	UC-03 – Save a Favorite Recipe
Primary actor	User
Supporting actors	To be confirmed once favorites storage is decided.
Preconditions — what must be true before it starts	The user is viewing a recipe in Cooket. The favorites feature is available. The user is logged in
Trigger — what starts it	The user selects the recipe's favorite button.
Main success scenario — numbered steps	1. The user selects the favorite button. 2. Cooket checks whether the recipe is already saved. 3. Cooket adds the recipe to the user's favorites. 4. Cooket confirms that the recipe has been saved.
Alternative and exception flows	2a. Already saved: Cooket indicates that the recipe is already in favorites without creating a duplicate. 3a. Saving fails: Cooket displays an error and allows the user to retry.
Postconditions — what is true when it succeeds	The recipe is saved and available through View Favorite Recipes.
Requirements realized	FR-05

3.5 Requirements to use case traceability

Requirement ID	Use case(s) that realize it	If blank — why, and what you will do
FR-01	UC-01 – Find Recipes Using Available Ingredients	-
FR-02	UC-01 – Find Recipes Using Available Ingredients	-
FR-03	UC-01 — Find Recipes Using Available Ingredients, optional popularity-sorting flow	-
FR-04	UC-02 — View Recipe Details	-
FR-05	UC-03 — Save a Favorite Recipe; UC-04 — View Favorite Recipes	-
FR-06	-	Marked WON'T for this release. Retain for future consideration; no current use case is assigned.
FR-07	UC-05 — Watch a Recipe Video	-
FR-08	-	Marked WON'T for this release. Retain for future consideration; no current use case is assigned.

What the matrix revealed, and what you changed

The matrix revealed that all functional requirements, whether planned or proposed for inclusion in this release, have use cases associated with them, and all use cases are associated with a functional requirement. FR-05 was mapped to both the save favorite and view favorites use cases because the two are needed for satisfying its acceptance criterion. FR-06 and FR-08 could not be mapped since they are won't be done for this release.

because of it:

Because of these findings, we linked FR-05 to both saving and viewing favorites and explicitly identified FR-06 and FR-08 as excluded from the current release. Popularity sorting, measurement units, and favorites storage still require clarification.

Review and updating for 1 - 3 needs to be done still

4 Use-Case Analysis

Chapter 7, first half. Here you take the use cases from section 3.4 and write the most important of them out turn by turn, still with the technology switched off.

4.1 Use cases selected for analysis

Use case	Name	Why it was selected	Non-trivial steps
UC-01	Find Recipes Using Available Ingredients	Core functionality; involves ingredient matching, recipe retrieval, and visible retrieval failures.	Step 2: validate criteria, retrieve recipes, and rank matches.
UC-03	Save a Favorite Recipe	Expanded at P1; involves duplicate detection, saving, and failure handling	Step 1: identify the recipe, check existing favorites, and save it.
UC-03	View Recipe Details	Lets users inspect ingredients, quantities, and instructions before cooking.	Step 1: obtain the selected recipe's details and handle unavailable information.

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Analyze at least three use cases, including the two you specified in full at P1. We recommend that one of them involves an external system or a failure that the actor can see.

Choose by risk and value, not by ease: the use case that carries most of the product, the one that touches most of your quality requirements, the one you are least sure you can build.

Budget: a quarter of a page.

4.2 Expanded use cases

Copy the block below once for each use case in 4.1.

UC-01 Find Recipes Using Available Ingredients

Primary actor	User
Supporting actors and external systems	External recipe sources
TUCBW (this use case begins when)	The user starts an ingredient search.
TUCEW (this use case ends when)	The user sees matching recipes, a no-results message, or a search-failure message.
Preconditions	The user can access Cooket. No account is required.
Postconditions	On success, matching recipes are displayed for the submitted criteria. Otherwise, the user receives an explanation and can revise or retry the search.
Traces to requirements	FR-01, FR-02; QR-01, QR-02, QR-03, QR-04; CON-01, FR-09

Actor input / action	System response
-	**(0)** Cooket offers an ingredient search, strict or flexible mode, and optional preferences.
(1) The user supplies ingredients, selects a mode, and optionally supplies flavor, food type, and texture preferences.	(3) Cooket accepts the criteria and makes them available for review.
(2) The user submits the search.	(4) Cooket indicates that the search is in progress, then displays matching recipes with their names, source references, and ingredient match scores.

Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1	2	No usable ingredients were supplied.	Cooket asks for at least one ingredient and retains the entered criteria.	Rejoins at step 1.
A2	2	No recipes match	Cooket displays a no-results message and allows the user to revise the criteria.	Ends; a revised search starts at step 1.
A3	2	Recipe retrieval fails.	Cooket reports the failure and allows a retry without re-entering the criteria.	Ends; retry rejoins at step 2.
A4	2	Some sources fail, but usable results remain.	Cooket displays the available matching recipes.	Ends with available results.

UC-03 Save a Favorite Recipe

Primary actor	user
Supporting actors and external systems	None specified at analysis level.
TUCBW (this use case begins when)	The user asks to save a recipe as a favorite.
TUCEW (this use case ends when)	The user sees confirmation that the recipe is saved, already saved, or could not be saved.
Preconditions	The user is viewing a recipe in Cooket. No account is required.
Postconditions	On success, the selected recipe appears once in the user's favorites and is available through UC-04. On failure, Cooket does not claim that saving succeeded.
Traces to requirements	FR-05; QR-04; CON-02

Actor input / action	System response
-	**(0)** Cooket displays a recipe and offers the option to save it as a favorite.
(1) The user asks to save the recipe.	(2) Cooket checks whether it is already saved. If not, Cooket saves it and confirms that it is available in favorites.

Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1	1	The recipe is already saved	Cooket indicates that it is already a favorite and creates no duplicate.	Ends with the existing favorite unchanged
A2	1	Saving fails	Cooket reports that the recipe could not be saved and allows a retry.	Ends; retry rejoins at step 1.

UC-02 View Recipe Details

Primary actor	User
Supporting actors and external systems	External recipe sources when details must be retrieved.
TUCBW (this use case begins when)	The user asks to view a recipe's details.
TUCEW (this use case ends when)	The user sees the recipe details or an explanation that details are unavailable.

Preconditions	Cooket displays a recipe reference in search results or favorites. No account is required.
Postconditions	On success, the recipe's title, ingredients with recorded quantities and units, ordered instructions, and source reference are visible. Missing information is identified rather than invented.
Traces to requirements	FR-04; QR-04; CON-01, CON-02

Actor input / action	System response
—	**(0)** Cooket displays recipe references and offers access to their details.
(1) The user selects a recipe and requests its details.	(2) Cooket displays its title, ingredients with quantities and units recorded by the source, ordered instructions, and original source reference.

Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1	1	Some details, quantities, or units are missing.	Cooket displays available information, identifies unavailable details, and provides the original source reference when available.	Ends with partial details.
A2	1	Recipe details cannot be retrieved.	Cooket reports that details are unavailable and allows a retry or return to the recipe list.	Ends; retry rejoins at step 1.

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

The rules from Lesson 13, all of which are checked:

Step (0) is what the actor sees before touching anything. It is often skipped, and it is where many missing requirements are found.

One turn per row. An actor action on the left, the system response on the right. Two steps merged into one row become a missing message on your sequence diagram in section 5.

End with an acknowledgment the actor can perceive. No acknowledgment, no TUCEW, no use case.

No background processing. If the actor neither sees it nor waits for it, it belongs to another use case.

Precondition or alternate flow? If it can be false while the system works normally, it is a flow you must handle, not an assumption.

No technology yet. No screen widgets, no database, no framework names. "Taps the green button" and "queries the PostgreSQL table" both fail the analysis check.

Every item the actor column names should already be an attribute in your domain model (5.1). Where it is not, you have found one.

Budget: about one page per use case.

EXAMPLE (COMETPARK): DELETE BEFORE SUBMITTING

UC-02 Find an Open Space · Primary actor: Driver · External system: Lot Sensor Network · TUCBW the driver asks for parking near a destination building · TUCEW the driver sees the recommended lot with its number of open spaces.

(0) The system shows the campus lots with the time of the latest occupancy update. (1) The driver chooses a destination building and permit type. (2) The system shows the three nearest lots the permit allows, each with open spaces and walking time. (3) The driver selects a lot. (4) The system shows the lot entrance and confirms the count is no older than 60 seconds.

A1 at step (2): no permitted lot has an open space · the system shows the nearest lot with expected turnover and the visitor option · ends.

4.3 Self-review log

Use case	Check from the Lesson 13 review checklist	Defect found	How it was fixed
UC-01	Preconditions versus alternate flows	The P1 precondition assumed recipe data was available, excluding a normal search failure.	Removed availability as a precondition and added flows for no results, retrieval failure, and partial source failure.
UC-01	Step (0) describes what the actor initially sees	The P1 flow started with ingredient entry without describing the initial system state.	Added step (0), showing the ingredient search, mode selection, and optional preferences.
UC-03	The use case ends with a visible acknowledgment	Saving needed explicit outcomes for success, duplicates, and failure.	Added confirmation for a saved favorite, an already-saved message, and a failure message with retry.
UC-03	Analysis avoids implementation details	Favorites storage has not been decided.	Described described saving favorites to the user’s account without naming a storage technology
UC-02	Missing information is handled as an alternate flow	The main flow alone did not explain incomplete or unavailable recipe details.	Added alternate flows for partial details and retrieval failure.
All three	One actor/system turn per row; clear start, end, and postconditions	No additional defect found in the reviewed draft.	Checked each row and confirmed that each outcome ends with information the user can perceive.

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Run the review checklist from Lesson 13 on each expanded use case before anyone draws a diagram, and record what each check found, including any defects you fixed. Report honestly; a clean result is acceptable when it is true. Budget: a quarter of a page.

5 Analysis Models

Chapter 7, second half. What has to exist inside the system for the use cases in section 4 to happen, and how those parts talk to each other.

5.1 Domain model

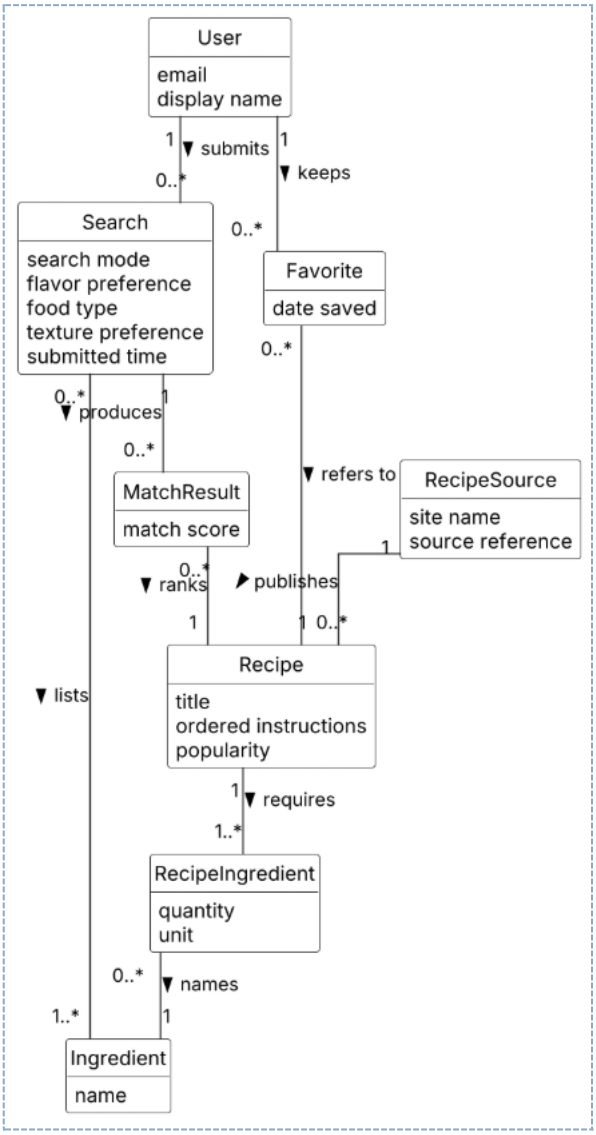


Figure 5.1 Domain model (conceptual classes, attributes, associations with multiplicities)

Class	What it represents (one line)	Key attributes	Associations (with multiplicity)
User	Someone with an account in our database	email, display name	User 1 — 0..* Search (submits); User 1 — 0..* Favorite (keeps)

Class	What it represents (one line)	Key attributes	Associations (with multiplicity)
Search	One set of criteria a user submits to find recipes	search mode (strict or flexible), flavor preference, food type, texture preference, submitted time	Search 0..* — 1..* Ingredient (lists); Search 1 — 0..* MatchResult (produces)
Ingredient	A food item, under one normalized name	name	Ingredient 1 — 0..* RecipeIngredient (named by); Ingredient 1..* — 0..* Search (listed in)
Recipe	An existing recipe collected from a recipe website	title, ordered instructions, popularity	Recipe 1 — 1..* RecipeIngredient (requires); Recipe 0..* — 1 RecipeSource (published by)

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Conceptual classes only: things and records from the problem domain, with attributes and associations. No operations, no data types, no database tables, no screen classes.

Every item the actor column of section 4.2 mentions is an attribute somewhere in this model. Every association has a multiplicity at both ends.

Budget: one page including the diagram.

EXAMPLE (COMETPARK): DELETE BEFORE SUBMITTING

Lot (name, zone, capacity) 1 ── 0..* Space (number, type) ; Space 1 ── 0..* OccupancyReading (time, state) ; Permit (type, valid until) * ── 1..* Lot "is valid in" ; Building (name) * ── * Lot "is near" (walking time).

5.2 Object interaction models

Interaction objects:

- **SearchPanel**: the object that runs UC-01. It receives the user's criteria, coordinates the other objects, and shows the results. It is in control for the whole interaction.
- **RecipeCatalog**: the collection of all stored recipes; it answers which recipes use which ingredients.
- **MatchScorer**: applies the strict or flexible mode and computes the match score for each candidate recipe.
- **RecipePanel**: the object that runs UC-03. It shows one recipe and handles the request to save it. It is in control for the whole interaction.
- **FavoritesList**: the collection of one user's Favorite records.

5.2.1 UC-01 Finding Recipes Using Available Ingredients

(a) Non-trivial steps.

- Step (4), the system response to the submitted search, is non-trivial. Cooket must record the search, find candidate recipes, apply the strict or flexible mode, score and rank the candidates, and display them.
- Step (0) is trivial: it only presents the search options.
- Step (1) is trivial: the user enters criteria and no object inside the system acts yet.
- Step (3) is trivial: it only shows the entered criteria back to the user.

(b) Scenario table.

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
1	User	submits search criteria	ingredients, search mode, flavor preference, food type, texture preference	SearchPanel
2	SearchPanel	records a new search for the user	User, ingredients, search mode, preferences, submitted time	Search
3	SearchPanel	asks for recipes that use any submitted ingredient	Ingredient names	RecipeCatalog
4	SearchPanel	asks to score and rank the candidates under the search mode	Search, candidate recipes with their RecipeIngredients	MatchScorer

(c) Analysis sequence diagram.

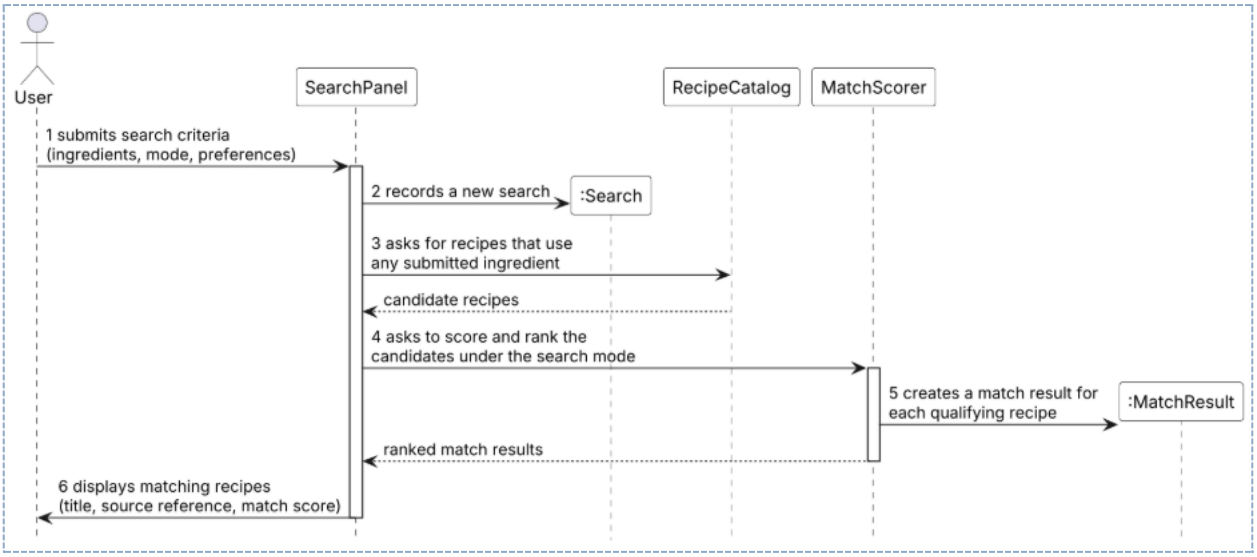


Figure 5.2.x Analysis sequence diagram for UC-xx; message numbers match the scenario table

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Write each step as a declarative sentence first, subject, verb, object, then fill the columns. Tables filled directly produce rows with no subject.

One message per table row, same numbers. Use the four translation cases from Lesson 14; the diagram is a conversion of the table, not a new drawing.

Messages are verb phrases, not operation signatures. "asks for the routes calling at the stop" is analysis; "getRoutes(stopId: int): List<Route>" is Chapter 9 and is not credited here.

Every object on the diagram is a class in 5.1 or an interaction object you name and explain in one line. One object is clearly in control of the interaction.

Budget: one to one and a half pages per use case.

EXAMPLE (COMETPARK): DELETE BEFORE SUBMITTING

Step 2 · Subject: LotFinder · Action: asks for the lots the permit allows · Data: permit type, destination · Object acted upon: PermitRules.

5.3 Consistency cross-check

Check	Result (pass, or what failed)	Fixed in
K1 Every item in an actor column (4.2) is an attribute in the domain model (5.1)	Pass. Ingredients → Ingredient.name; mode → Search.search mode; flavor, food type and texture → Search attributes; the selected recipe → Recipe; saving → Favorite. The domain model was built from the 4.2 actor columns.	—

Check	Result (pass, or what failed)	Fixed in
K2 Every object on a sequence diagram is a domain class or a declared interaction object	Pass. Search, MatchResult and Favorite are domain classes; SearchPanel, RecipeCatalog, MatchScorer, RecipePanel and FavoritesList are declared in 5.2.	—
K3 Every message on a sequence diagram corresponds to exactly one scenario-table row	Pass. UC-01 has 6 numbered messages and 6 rows; UC-03 has 5 numbered messages and 5 rows, numbered identically. Returns are unnumbered.	—
K4 Every non-trivial step in 5.2 (a) has at least one scenario-table row	Failed: in 4.2 the UC-01 steps were numbered out of order ((3) appeared before (2)), so "step (4)" could not be matched to a turn unambiguously. Once renumbered, UC-01 step (4) maps to rows 2–6, and UC-03 step (2) maps to rows 2–5.	4.2: UC-01 steps renumbered (0)–(4) in turn order.
K5 Every analyzed use case traces to at least one functional requirement	Failed: after CH-03, the 4.2 "Traces to requirements" lines still cited the withdrawn CON-02 and not FR-09.	4.2: CON-02 replaced with FR-09. UC-01 → FR-01, FR-02, FR-09; UC-03 → FR-05, FR-09; UC-02 → FR-04, FR-09.
K6 The last messages of each diagram make the use case postcondition true	Pass. UC-01 message 6 displays the matching recipes; UC-03 messages 3–5 store the favorite once and confirm it to the user.	—

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Run the six checks across the work of different authors; they are designed to catch the seams between people. A failed check that you found and fixed is more convincing than a column of passes. Budget: a quarter of a page.

6 High-Level Design (Architecture)

Chapter 8. The rule now reverses: a high-level design that does not say where things run, what they are built from and what that choice cost has not been done.

6.1 Architectural drivers: quality-attribute scenarios

QAS	Quality attribute	Stimulus	Artifact	Response	Response measure	From
QAS-01	Performance	A logged-in user submits a search with 1–10 ingredients under normal load	Search API, Recipe Finder and Match Scorer	Cooket returns ranked matching recipes to the Web Client	Within 3 s for at least 95 of 100 searches	QR-01
QAS-02	Usability	A first-time user opens Cooket and wants to search by ingredients, wants no help	Web Client	The user completes an ingredient search and sees results	At least 9 of 10 first-time users finish within 60 s	QR-02
QAS-03	Correctness	A user's typed ingredient is compared with a scraped recipe's ingredient line	Match Scorer	The ingredient is correctly reported as matched or missing	At least 90% correct on a test set of 100 manually verified comparisons	QR-03

Constraints that shape the architecture:

- **CON-01** (retrieve, never generate, recipes) is why a Recipe Finder reads existing recipe pages and no component writes recipes.
- **FR-09** (CH-03: accounts are required before searching) puts an Account & Favorites Service in front of every search and favorites request.

2.4 Constraints: No payments, so no payment provider or card data enters the design.

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

At least three scenarios, preferably covering more than one quality attribute (for example performance, availability, security, modifiability, usability). Derive them from your P1 quality requirements and say which.

The response measure is the part teams leave out, and it is the only part that lets anyone tell whether the architecture worked. A scenario without a number is an opinion.

Below the table, list in two or three lines the constraints (CON IDs) that shape the architecture.

Budget: half a page.

EXAMPLE (COMETPARK): DELETE BEFORE SUBMITTING

QAS-01 · Performance · Stimulus: 400 drivers request lot status during the 7:45 to 8:15 AM peak · Artifact: the occupancy service · Response: returns current open-space counts · Measure: within 2 s for 95% of requests · From QR-02.

6.2 Architectural style and alternatives

Style considered	Drivers it serves	What it costs	Verdict
Layered client–server (browser presentation, Node.js application layer, Supabase data)	QAS-04: API keys, scraping and session checks stay on the server, and the client never decides what the server could. QAS-03: matching rules live in one server component that can change without touching the client. QAS-01: one place to add caching later.	The server is a single point of failure, and every request crosses an extra network hop; the thin client can do nothing offline.	Chosen
Pipe-and-filter (search → extract → clean → score as one pipeline for the whole system)	QAS-03 and QAS-05: each filter is easy to test, and a failed page can be dropped by one filter.	It suits data flow, not interaction. Login and saving favorites are request–response interactions with nobody "downstream". We keep the pipeline idea only inside the Recipe Finder and Match Scorer.	Rejected
Repository (scrape recipes ahead of time into one shared Supabase store that all components read)	QAS-01: searches would read a local store instead of live websites.	The store becomes a single point of failure and everyone's schema problem; results are only as fresh and as broad as the last scrape; it risks the A5 free-tier limit.	Rejected for now; revisit if the spike shows live scraping misses QAS-01

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Name the style (or combination) you chose and at least two you rejected, using the vocabulary from Lessons 15 and 16: layered (N-tier), client-server, repository, model-view-controller, pipe-and-filter, event-driven, service-oriented or microservices.

One paragraph below the table explains the choice in terms of the scenarios in 6.1. "We are most comfortable with it" is not a driver. ADR-01 in section 7 records this decision formally.

Budget: half a page.

6.3 Component view

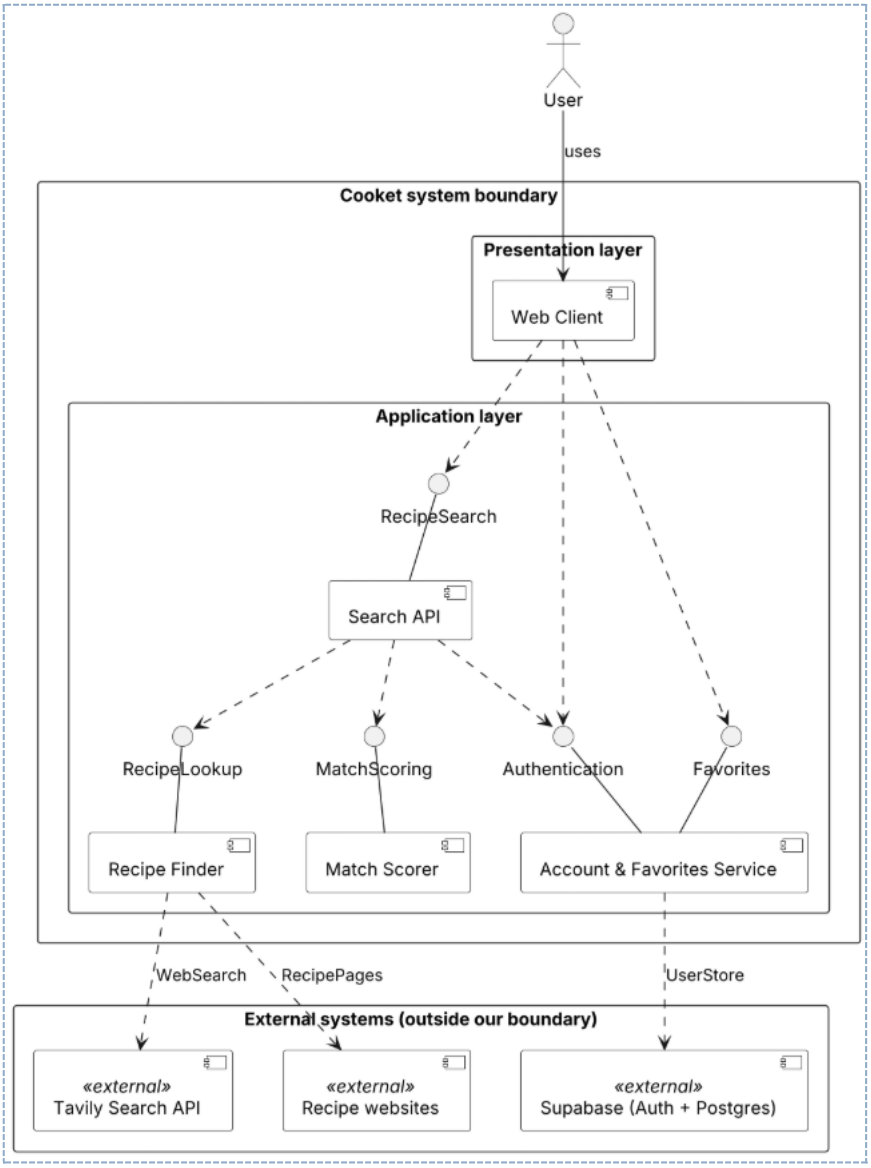


Figure 6.3 Component diagram: components, provided and required interfaces, external systems

Component	Responsibility (one sentence)	Provides	Requires	Answers to (UC / QAS / ADR)
Web Client	Lets the user log in, enter search criteria, and view, open and save recipes.	(user interface)	RecipeSearch, Authentication, Favorites	UC-01, UC-02, UC-03, UC-06; QAS-02
Search API	Accepts a search, checks the session and criteria, and returns ranked recipes.	RecipeSearch	Authentication, RecipeLookup, MatchScoring	UC-01; QAS-01, QAS-04; ADR-01

Component	Responsibility (one sentence)	Provides	Requires	Answers to (UC / QAS / ADR)
Recipe Finder	Finds candidate recipe pages for the ingredients and extracts each recipe's title, ingredients, steps and source.	RecipeLookup	WebSearch (Tavily), RecipePages (recipe websites)	UC-01, UC-02; QAS-05; CON-01
Match Scorer	Applies strict or flexible mode, matches ingredients, and ranks recipes by match score.	MatchScoring	—	UC-01; QAS-03

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Point at any box and say which use case, which quality scenario or which decision put it there. A box with no answer cannot be credited.

Show external systems outside the boundary. Name interfaces by what they offer ("OccupancyQuery"), not by operation signatures, which are Chapter 9.

Budget: one page including the diagram.

6.4 Deployment view

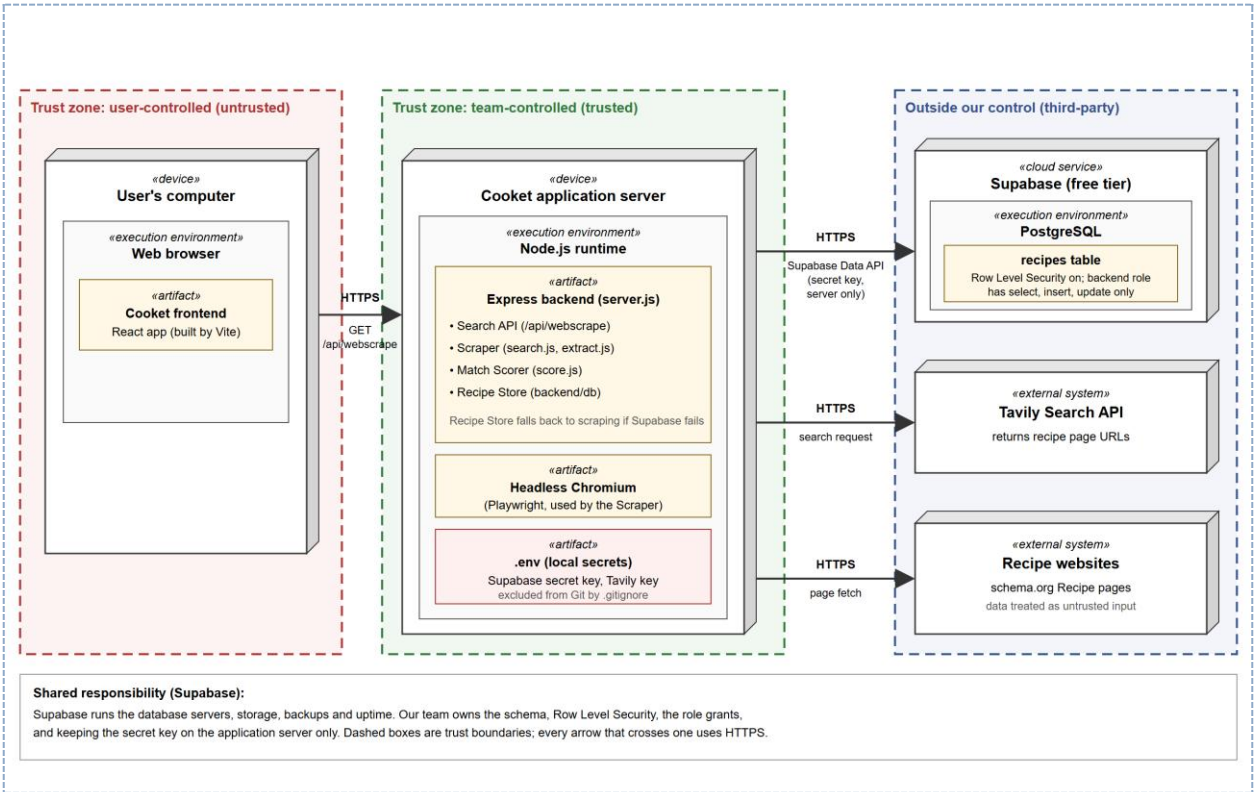


Figure 6.4 UML deployment diagram: nodes, execution environments, deployed artifacts, communication paths with protocols, trust boundaries

Node	What runs on it (artifacts, components)	Communicates with (protocol)	Trust zone
User's computer (web browser)	Cooket frontend (React app built by Vite)	Cooket application server (HTTPS, GET /api/webscrape, JSON results)	User-controlled, untrusted
Cooket application server (Node.js runtime)	Express backend with the Search API, Scraper, Match Scorer and Recipe Store; headless Chromium (Playwright) for the Scraper; .env file with the Supabase and Tavily keys	Web browser (HTTPS); Supabase (HTTPS, Supabase Data API); Tavily Search API (HTTPS); recipe websites (HTTPS)	Team-controlled, trusted
Supabase (managed PostgreSQL, free tier)	recipes table, with Row Level Security on and the backend role allowed only select, insert and update	Cooket application server only (HTTPS)	Third-party managed (shared responsibility)

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Every component in 6.3 is deployed somewhere in 6.4. Every communication path names its protocol. Mark the trust boundaries: where data crosses from a zone you control into one you do not.

If you use a cloud service, say which responsibilities stay with your team under the shared-responsibility model.

Budget: one page including the diagram.

6.5 From analysis to architecture

Analysis object (from 5.1 and 5.2)	Component that owns it (6.3)	Note (split, merged, external)
User	Account & Favorites Service	Credentials are held by Supabase Auth (external); the service owns the session check.
Search	Search API	Exists for one request; not stored at this release.
Ingredient	Match Scorer	Normalized names (plurals, quantities removed) are produced here.
Recipe	Recipe Finder	Extracted per search from the source page; not stored in bulk (A5).
RecipeIngredient	Recipe Finder	Extracted with its recipe; quantity and unit kept as the source records them (FR-04).

RecipeSource	Recipe Finder	The source reference (URL) travels with each recipe; the websites themselves are external.
MatchResult	Match Scorer	Includes the match score and the matched and missing ingredients.
Favorite	Account & Favorites Service	Persisted in Supabase per account (Q4).
RecipePanel	Split: Web Client and Account & Favorites Service	The display part runs in the Web Client; the duplicate check and save run on the server.
RecipeCatalog	Recipe Finder	Merged: realized by a live web search plus page extraction rather than a stored catalog.
MatchScorer	Match Scorer	One-to-one.
FavoritesList	Account & Favorites Service	One-to-one; backed by the favorites table in Supabase.

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Every class and interaction object from section 5 lands in exactly one component, or the note says why it was split, merged or left to an external system. This table is how the reader sees that the architecture realizes the analysis rather than replacing it. Budget: a quarter of a page.

6.6 Design principles applied

Principle		Where it shows in your design	Evidence (component, interface, ADR)
Separation of concerns	Handling the HTTP request, finding recipes, and scoring them are three separate components.	Search API (`routes/webscrape.js`), Recipe Finder (`scraper/search.js`, `extract.js`), Match Scorer (`scraper/score.js`)	
Information hiding	The Recipe Finder hides which search provider is used and how pages are parsed (schema.org JSON-LD); callers only see RecipeLookup.	RecipeLookup interface; replacing Tavily changes only `search.js`	
Low coupling	The Web Client depends only on three server interfaces and never calls Tavily, recipe sites or Supabase directly.	RecipeSearch, Authentication, Favorites; Figure 6.3	

Principle		Where it shows in your design	Evidence (component, interface, ADR)
High cohesion	The Match Scorer has one responsibility: deciding how well a recipe matches the ingredients. The plural-matching fix touched only this component.	Match Scorer; commits e92275a and 08a65c9 (PR #14) changed only `score.js`	
Design for change	The way recipes are found is the most likely part to change (keyword query today, possibly a smarter query later), so it sits behind an interface.	RecipeLookup; `buildQuery` in `search.js` is isolated for replacement	
Keep it simple (KISS)	One Node.js server and a managed database instead of separately deployed services or our own database server.	Figure 6.4; microservices rejected in 6.2	

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

One line per principle, pointing at something concrete. A principle you cannot point at in your own design should be left blank and explained in one sentence, not filled with a definition from the slides. Budget: a third of a page.

7 Design Rationale

7.1 Architecture decision records

At least three. ADR-01 records the style decision from 6.2. Use the format you practiced in the Lesson 16 workshop, written about your own project. Copy the block for each decision.

ADR-01: Use a Layered Client-Server Architecture

Status	Proposed – pending team approval
Date and owner	Nov 8, 2026 – Raagavika Govindarajan
Context (drivers)	Cooket requires a system that separates user interaction, recipe processing, and data storage. QAS-01 requires recipe searches to return results within 3 seconds for at least 95% of searches. QAS-03 requires secure HTTPS communication.
Options considered	1. Layered client-server architecture 2. Microservices architecture 3. Monolithic architecture
Decision	Use a layered client-server architecture separating the frontend, backend processing, and database responsibilities.
Consequences	This improves maintainability and separates responsibilities. However, communication between layers introduces additional network requests and potential delays.
Evidence	Existing frontend, backend scraper integration, and Supabase dependency described in the project document. Performance validation is not yet available.
Related	QAS-01, QAS-03, UC-01, FR-01, FR-02.

ADR-02: Use a Stored Recipe Catalog for Searching

Status	Proposed – pending team approval
Date and owner	Nov 8, 2026 – Raagavika Govindarajan
Context (drivers)	Cooket retrieves existing recipes from external websites. QAS-01 requires fast search results, while QAS-02 requires at least 90% ingredient matching accuracy.
Options considered	1. Scrape recipe websites during every search 2. Store collected recipes in a database and search those records.
Decision	Store scraped recipes in a recipe catalog and perform ingredient matching against the stored records.

Consequences	This reduces dependence on external websites during searches and supports faster responses. However, stored recipes can become outdated and require periodic updates.
Evidence	Section 2.5 identifies Supabase as a database dependency. Section 5 identifies RecipeCatalog and MatchScorer. No measured performance result is documented yet.
Related	QAS-01, QAS-02, CON-01, UC-01, FR-02, QR-03.

ADR-03: Require Authantication before Recipe Searches

Status	Proposed – pending team approval
Date and owner	Nov 8, 2026 – Raagavika Govindarajan
Context (drivers)	FR-09 requires users to register or log in before searching. QAS-03 requires all browser-to-server communication to use HTTPS. User accounts also support personalized features such as saved favorites.
Options considered	1. Allow anonymous searches and require login only for favorites. 2. Require authentication before accessing recipe searches.
Decision	Require users to authenticate before searching for recipes and use HTTPS for browser-to-server communication.
Consequences	This supports account-based features and consistent access control. However, mandatory registration adds steps before users can access the main functionality.
Evidence	FR-09 and change CH-03 document the authentication requirement. QR-04 specifies HTTPS communication. Implementation verification is still required.
Related	FR-09, FR-05, QR-04, QAS-03, CH-03.

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

A decision without a cost is a preference. If you cannot say what you gave up, you have not finished the record.

Options must be real alternatives the team discussed, not a straw man added to fill the row.

Budget: about a third of a page per ADR.

EXAMPLE (COMETPARK): DELETE BEFORE SUBMITTING

ADR-02 · Push occupancy changes to clients rather than polling · Context: QAS-01, CON-02 (campus Wi-Fi limits) · Options: client polling every 10 s; server push over a persistent connection · Decision: server push · Consequences: favors performance and freshness; costs connection management on the occupancy service and a fallback when push fails (A2 in UC-02) · Evidence: spike S-01.

7.2 Trade-off summary

Quality favored	Architectural move	Cost accepted	Recorded in
Maintainability	Separate frontend, backend processing, and database into layers.	Additional communication between layers increases complexity and potential latency.	ADR-01
Performance	Search stored recipe records instead of scraping external websites for every request.	Recipes require periodic updates to remain accurate and current.	ADR-02
Security and access control	Require user authentication before recipe searches and use HTTPS.	Users must complete registration or log in before using the primary search functionality.	ADR-03

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

One row per real trade-off. This is the table a reviewer reads first. Budget: a quarter of a page.

8 Prototype Evidence: Architecture Spike

Spike ID and title	S-01
Risk or decision it addresses	R1, R6 / ADR-02 / QAS-01
Question (answerable yes or no)	Yes
Set-up	A Node.js script (spikes/S-01/measure.js) that uses our real backend code: Tavily search, Playwright headless Chromium for extraction, and our Supabase free-tier project (US region) with the real recipes table. Real recipe websites were scraped, nothing was simulated. It was run on a Windows laptop over home internet on November 6, 2026. The keys stay in backend/.env, which is not committed.
How it was measured	For three searches ("chicken, rice, garlic", "flour, egg, milk", "pasta, tomatoes, garlic") the script timed one live scrape and saved the recipes. It then timed the Supabase lookup 10 times, recording the median and worst time and whether there were at least 3 stored recipes using every ingredient (enough to skip scraping). Then the full search function was run once more for each search to time it end to end.
Result (numbers)	Result (numbers) Database lookup: median 0.10 to 0.11 s, worst 0.21 s. Live scrape: 1.95 s, 3.45 s and 4.92 s (up to 5.7 s on earlier first runs). End to end, "pasta, tomatoes, garlic" returned 4 recipes from Supabase in 0.24 s with no scraping, while "chicken, rice, garlic" didn't have enough stored recipes yet and scraped live in 2.07 s. Only 1 to 3 of the 10 pages per search gave a usable recipe (many sites returned HTTP 403), so only 1 of the 3 searches had enough stored recipes to skip scraping.
What it changed or confirmed	Confirmed ADR-02: searches answered from Supabase meet QAS-01 / QR-01 easily, while live scrapes don't reliably meet it, so storing recipes is needed and not optional. It also showed a new limitation: how often the cache helps depends on how many recipes the scraper keeps (R6), which is written up in docs/Database.md.
Where to see it	ommit 744fb1b, /spikes/S-01/README.md (results table and conclusion), /spikes/S-01/measure.js

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Build the smallest thing that answers your riskiest architectural question, run it, and measure it. Time-box it to a few hours. It may be thrown away afterwards; it is not the product.

Good spikes answer questions such as: can we read the external data source, and is it fresh to within 60 seconds? Does the chosen hosting meet QAS-01 under a simulated peak? Does the authentication provider fit our trust boundary?

Never commit passwords, API keys or other credentials; keep them in local configuration excluded from the repository.

The code, the measurement and a short README are committed to /spikes before the deadline. A spike described but not in the repository is not evidence.

Budget: half a page.

9 Traceability

9.1 Extended traceability matrix

Requirement	Use case	Analysis classes	Component(s)	QAS / ADR	Evidence
FR-01	UC-01	User, Search, Ingredient	SearchPanel	QAS-01, ADR-01	Frontend ingredient input
FR-02	UC-01	Search, Recipe, MatchResult	RecipeCatalog, MatchScorer	QAS-01, QAS-02, ADR-02	Recipe search implementation
FR-03	UC-01 (optional)	Recipe, MatchResult	MatchScorer	ADR-02	Not yet verified
FR-04	UC-02	Recipe, RecipeIngredient, Ingredient	RecipeCatalog	ADR-02	Recipe details analysis
FR-05	UC-03, UC-04	User, Favorite, Recipe	Favorite component (proposed)	ADR-01, ADR-03	UC-03 analysis
FR-06	Not assigned (WON'T)	-	Not assigned	-	Excluded from current release
FR-07	UC-05	Recipe, RecipeSource	RecipeCatalog (proposed)	CON-01	Not yet verified
FR-08	Not assigned (WON'T)	-	Not assigned	-	Excluded from current release
FR-09	UC-01 (precondition)	User	Authentication component (proposed)	QAS-03, ADR-03	CH-03
QR-01	UC-01	Search, matchResult	Search, MatchResult	QAS-01, ADR-02	Performance test pending
QR-02	UC-01	Search	Search	ADR-01	Usability test pending
QR-03	UC-02	Ingredient, MatchResult	Ingredient, MatchResult	QAS-02, ADR-02	Matching accuracy test pending
QR-04	UC-01, UC-02, UC-03	User	User	QAS-03, ADR-03	HTTPS verification pending

Requirement	Use case	Analysis classes	Component(s)	QAS / ADR	Evidence
CON-01	UC-01, UC-02	Recipe, RecipeSource	Recipe, RecipeSource	ADR-02	Existing-recipe retrieval design
CON-02	Withdrawn (CH-03)	-	-	CH-03	Change record in Section 0.2

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Extend the P1 matrix one column at a time: from each requirement to the use case, to the analysis classes that realize it, to the component that will own it, to the decision or scenario that shaped it, and to whatever evidence exists (spike, measurement).

Requirements realized by the use cases you analyzed need the full chain. Every other MUST requirement needs at least its component. Withdrawn requirements stay in the matrix, marked as such.

Budget: one page.

9.2 Gap analysis

Gap	Items	What you did about it
Requirements with no component	FR-06 and FR-08 are marked WON'T. FR-05 and FR-09 require confirmation of their component ownership.	Excluded FR-06 and FR-08 from the current release. Proposed favorites and authentication components for FR-05 and FR-09, pending confirmation in Section 6.3.
Components with no requirement, QAS or ADR	Cannot be determined until the Section 6.3 component diagram is completed.	Identified the need to cross-check every component against a requirement, quality scenario, or architecture decision before finalizing the design.
Use cases not analyzed (and why that is acceptable)	UC-04 (View Favorite Recipes) and UC-05 (Watch a Recipe Video).	Prioritized UC-01, UC-02, and UC-03 for detailed analysis because they cover core recipe searching, recipe details, and saving favorites. The remaining use cases retain their requirement mappings for future design work.

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

An empty gap table is believable only if the matrix above is complete. Say what each gap means and what you did. Budget: a quarter of a page.

10 Project Plan (updated; was P1 section 4)

10.1 Roles and working agreement

> **GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING**
> Keep your P1 text and record only what changed. If roles moved between members, say so here and in 0.2.

10.2 Plan to P3

Milestone	Date	Owner	Done when
Fully complete login/registration system	11/8/26	Gabriel	12/1/26

10.3 Risks (updated)

Risk	Status since P1	Likelihood / impact	Mitigation and owner
R1 Supabase outage or connection limits	Unchanged	Medium / High	If Supabase drops connection, the app will show the last cached results and give the user a retry button. Owner: Database Owner
R2 Inconsistent scraped ingredient names	Open, now addressed in design	High / High	Raw ingredient text can be messy, so we're adding a quick text-normalization step before saving, plus a spike to test how well it matches. Owner: Scraper Owner
R3 Target sites change their HTML	Downgraded from Medium / Medium	Low / Medium	Websites might change their layouts, so we'll pull from the embedded schema.org data instead of HTML tags and add a quick filter to drop bad records. Owner: Scraper Owner
R4 Scope creep	Unchanged	Medium / Medium	Keep everyone aligned on feature limits and run new ideas by the team first. Owner: Whole Team

Risk	Status since P1	Likelihood / impact	Mitigation and owner
R5 Frontend and database integration delays	Open	Medium / High	Wiring the frontend and backend could bottleneck us, so we'll agree on the data schema early and test using mock data. Owner: Whole Team
R6 Target sites block the scraper or forbid scraping	New, from assumption	Medium / High	Respect each site's robots.txt and terms, rate-limit requests, and keep at least three target sites so losing one doesn't stop database population. Owner: Scraper Owner
R7 Supabase free-tier limits exceeded	New, from assumption	Low / Medium	Cap the number of stored recipes and monitor usage on the Supabase dashboard. Owner: Database Owner
R8 Favorites lost if the user clears browser data	Closed by CH-03: favorites are stored per account	Medium / Low	Provide an optional JSON export/import feature for favorites so users can manually backup their favorites before clearing browser storage. Owner: Frontend Developer

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Close or downgrade P1 risks with a reason. Add the architectural risks your design created, and point to the spike or ADR that addresses each.

10.4 Repository

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

State the repository URL, the folder layout (/docs/p2, /docs/p2/diagrams, /spikes), your branch and pull-request rule, and confirm that the tag p2 points at the commit containing this PDF.

Access: the repository is public, or the instructor account announced on Canvas has read access before the deadline. A repository that cannot be opened at grading time cannot earn repository or contribution credit.

Repository URL:

https://github.com/zxahamza20/CS3354-Project-Cooket. The repository is public, so it can be opened by anyone at any time.

Folder layout:

/docs/p1 holds the P1 PDF, /docs/p2 holds this PDF, /docs/p2/diagrams holds the source file of every diagram for P2, /spikes/S-01 will hold the spike and its measured results.

Branch and pull-request rule:

Each member works on a feature branch named after the section or component they own, and merges into main through a pull request that at least one other member reviews. Nobody commits directly to main. The exception is repository setup, such as creating folders or the .gitignore, which may be committed to main. Each member commits their own sections from their own GitHub account, so the history shows who wrote what. GitHub tracks tasks and owners.

Credentials:

Database keys (our Supabase secrets) stay in a local .env file that .gitignore excludes, so that no credentials are committed and leaked.

Tag:

The tag p2 points at the commit that contains this exact PDF.

10.5 Use of AI tools in P2

Member	Tool	What it was used for	What you checked or changed
Gabriel Aguilar-Vega	Claude	Checked that all diagrams I made aligned with the doc correctly. Also insured no other instances of CON-02 came up in the doc and were changed it if	Added FR-09

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Declare every use, including code assistants used in the spike. Commit messages carrying an AI co-author line must be matched by a row here. Using AI is permitted; using it without declaring it is not. Write "None" if no tool was used.

11 Open Questions (updated; was P1 section 5)

ID	Question	Raised by	Owner and due date	Status or answer	Assumption meanwhile
Q1	How should ingredients be stored so typed names match scraped names?	David	Database Owner, before P3	Answered: each recipe links to normalized ingredient records, they are normalized when scraped	
Q2	What happens if a site changes layout and the scraper pulls bad data?	David	Scraper Owner, before the spike	Answered: parse schema.org recipe data, and reject records missing required fields	
Q3	How is the match score calculated?	David	Whole Team, before P3	Open	Score for each recipe is calculated by what ingredients the user has
Q4	Where are favorites stored?	David	Whole Team, before P3	Answered: in the database, per account.	
Q5	Which target sites will be scraped and do their terms allow it?	David	Scraper Owner, before the spike	Open	At least three sites that use schema.org recipe data
Q6	Which units of measurements do we use for recipes?	David	Requirements Owner, before P3	Open	US customary, such as ounces and pounds
Q7	What does "popular" mean from P1 FR-03?	David	Whole Team, before P3	Open	Using the source site's rating count where the recipe was published

GUIDANCE: DELETE THIS BOX BEFORE SUBMITTING

Close the P1 questions you answered and say how. Add the questions analysis and design raised: a step whose triviality was unclear, a scenario with no agreed response measure. If no questions remain open, say how the earlier ones were resolved.

12 Baseline v1.0 Declaration (was P1 section 6)

We declare this document, version 1.0, the baseline for P2. Sections 1 to 9 describe what we will build in P3. Any later change to them is recorded in section 0.2 with its reason.

Team member	Signature or initials	Date
David Vu	David Vu	November 8, 2026
Gabriel Aguilar-Vega	Gabriel Vega	November 8, 2026
Raagavika Govindarajan	Raagavika G	November 8, 2026

How P2 Is Marked

Delete this page and the next before submitting. 60 points: 48 for the shared team artifact (80%) and 12 for individual contribution evidence (20%).

Criterion	Points	What full marks look like
Baseline, change control, plan and repository (sections 0.2, 1 to 3, 10 to 12)	6	Every point of the P1 feedback is answered in 0.2. Requirement IDs are preserved and withdrawn items marked. The plan to P3, risks, AI-use declaration, open questions and signed v1.0 baseline are up to date. The repository is accessible and, before the deadline, holds this PDF, the diagram sources and the spike, with the tag p2 on that commit.
Use-case analysis (section 4)	8	At least three use cases expanded turn by turn with step (0), one turn per row, an acknowledgment at the end, real alternate flows, and no technology. Preconditions are true assumptions, not hidden alternate flows. The self-review log records what each check found, including any defects fixed.
Domain model and object interaction (section 5)	10	A conceptual domain model with attributes and multiplicities that contains every item the actor columns mention. At least two use cases carried through non-trivial steps, a five-column scenario table and an analysis sequence diagram whose messages match the table row for row. The six cross-checks are run honestly.
High-level design (section 6)	10	At least three quality-attribute scenarios with response measures. A named style chosen over at least two alternatives for reasons tied to those scenarios. A component view in which every component answers to a use case, scenario or decision; a UML deployment view with protocols and trust boundaries; every analysis object mapped to a component; design principles pointed out in the design itself.
Design rationale (section 7)	6	At least three decision records, each with real alternatives, a decision and a stated cost. The trade-off summary agrees with them.
Prototype evidence (section 8)	4	One spike that answers a real architectural question with a measured result, committed with its README before the deadline, and linked to the decision it changed or confirmed.
Traceability (section 9)	4	Every requirement realized by the analyzed use cases is traced through analysis classes to a component, and to the scenario or decision that shaped it where one exists. Every other MUST requirement reaches at least a component. Gaps are named, explained and acted on.
Document quality	up to −3	Deducted from the team points, not awarded: guidance, example or warning boxes left in, placeholders, sections out of order or renumbered, unreadable diagrams, or a document far beyond its length budget.

Criterion	Points	What full marks look like
Individual contribution evidence (section 0.1)	12 each	12: owns specific sections or artifacts, with own commits or pull requests before the deadline and at least one review of a teammate's pull request. 9: specific ownership and own commits, but thin or no review evidence. 6: specific claim with indirect evidence only (issues owned, or work uploaded explicitly on the member's behalf). 4: specific claim with no repository evidence. 2: vague or duplicated claim. 0: no credited work. A member with no credited work at all (absent from the contribution record and with no commits) may also lose the team portion, because the syllabus does not allow credit for work completed by others. This applies to individual non-participation only, never to a whole team.

WHERE P1 MARKS WERE LOST (DELETE THIS BOX BEFORE SUBMITTING)

Repositories that were empty, unreachable, or held a single upload from one account at the deadline. Repository and contribution credit could not be given, however good the document was.

Contribution records that cited commits, pull requests or files that were not in the repository. Everything cited is checked.

Guidance boxes, placeholders and unconfirmed draft text left in the submitted document.

Several members uploading different versions to Canvas. One member submits. If more than one file is uploaded, the latest upload is the one graded.

Quality requirements with no number in them, and requirements whose source was still "pending".

Work committed after the deadline. It is not counted, even a few minutes after.